

웹시스템개발1

콜백 함수



콜백 함수란?



- 다른 함수의 인자로 전달하는 함수로써, 그 제어권도 함께 위임한 함수

cbFunc 함수를 setInterval 함수의 첫 번째 인자로 전달,

setInterval 함수가 cbFunc 함수의 제어권을 넘겨 받음

setInterval 함수가 0.3초마다 cbFunc 함수를 호출

즉, cbFunc 함수의 호출 주체는 setInterval 함수

```
var count = 0;
var timer = setInterval(function() {
  console.log(count);
  if (++count > 4) clearInterval(timer);
}, 300);
```

```
var count = 0;
var cbFunc = function() {
  console.log(count);
  if (++count > 4) clearInterval(timer);
};
var timer = setInterval(cbFunc, 300);
```

콜백 함수란?



```
1
2  var newArr = [10, 20, 30].map(function (element, index, array) {
3      console.log(element, index, array);
4      return element + 5;
5  });
6
7  console.log(newArr);
8
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe ex.js

10 0 [10, 20, 30]

20 1 [10, 20, 30]

30 2 [10, 20, 30]

[15, 25, 35]

콜백 함수란?



● Array.prototype.map 함수의 구현

```
1
2  Array.prototype.map = function (callback, thisArg) {
3      var mappedArr = [];
4      for (var i = 0; i < this.length; i++) {
5          var mappedValue = callback.call(thisArg || global, this[i], i, this);
6          mappedArr[i] = mappedValue;
7      }
8      return mappedArr;
9  };
10
11  console.log([1, 2, 3].map(v => v + 1))
12
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe ex.js
[2, 3, 4]

콜백 함수는 함수다



- 콜백 함수로 어떤 객체의 메소드를 전달하더라도 그 메소드는 메소드가 아닌 함수로서 호출

```
1
2  var obj = {
3    method: function () {
4      console.dir(this, { depth: 0 })
5    }
6  };
7
8  obj.method();
9  [1].forEach(obj.method);
10
```

```
{ method: [Function: method] }
Object [global] {
  global: [Circular],
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function],
  queueMicrotask: [Function: queueMicrotask],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function]
}
```

콜백 함수 내부의 this에 다른 값 바인딩



● 전통적인 방식

```
1
2   var obj = {
3       name: 'obj',
4       func: function () {
5           var self = this;
6           return function () {
7               console.log(self.name);
8           };
9       }
10  };
11
12  var callback = obj.func();
13  setTimeout(callback, 1000);
14
```


콜백 함수 내부의 this에 다른 값 바인딩



● 화살표 함수 사용 방식

```
1
2   var obj = {
3       name: 'obj',
4       func: function () {
5           return () => {
6               console.log(this.name);
7           };
8       }
9   };
10
11   var callback = obj.func();
12   setTimeout(callback, 1000);
13
```

콜백 함수 내부의 this에 다른 값 바인딩



● bind 함수 사용 방식

```
1
2   var obj1 = {
3       name: 'obj1',
4       func: function () {
5           console.log(this.name);
6       }
7   };
8   setTimeout(obj1.func.bind(obj1), 1000);
9
10  var obj2 = { name: 'obj2' };
11  setTimeout(obj1.func.bind(obj2), 1500);
12
```


콜백 지옥과 비동기 제어



- 콜백 지옥 (callback hell)은 콜백 함수를 익명 함수로 전달하는 과정이 반복되어 코드의 들여쓰기 수준이 감당하기 힘들 정도로 깊어지는 현상
 - 주로 이벤트 처리나 서버 통신과 같은 비동기 작업 수행시 자주 등장
- 동기 (synchronous)
 - 현재 실행 중인 코드가 완료된 후에야 다음 코드 실행
- 비동기 (asynchronous)
 - 현재 실행 중인 코드의 완료 여부와 무관하게 즉시 다음 코드 실행

콜백 지옥과 비동기 제어

● 0.5초 주기마다 커피 목록을 수집하고 출력

에스프레소
에스프레소, 아메리카노
에스프레소, 아메리카노, 카페모카
에스프레소, 아메리카노, 카페모카, 카페라떼

```
1
2  setTimeout(
3      function (name) {
4          var coffeeList = name;
5          console.log(coffeeList);
6
7          setTimeout(
8              function (name) {
9                  coffeeList += ', ' + name;
10                 console.log(coffeeList);
11
12                 setTimeout(
13                     function (name) {
14                         coffeeList += ', ' + name;
15                         console.log(coffeeList);
16
17                         setTimeout(
18                             function (name) {
19                                 coffeeList += ', ' + name;
20                                 console.log(coffeeList);
21                             },
22                             500,
23                             '카페라떼'
24                         );
25                     },
26                     500,
27                     '카페모카'
28                 );
29             },
30             500,
31             '아메리카노'
32         );
33     },
34     500,
35     '에스프레소'
36 );
37
```

콜백 지옥과 비동기 제어

- 익명의 콜백 함수를
모두 기명 함수로 전
환
- 일회성 함수를 전부
변수에 할당해야 한다
는 단점

```
1
2  var coffeeList = '';
3
4  var addEspresso = function (name) {
5      coffeeList = name;
6      console.log(coffeeList);
7      setTimeout(addAmericano, 500, '아메리카노');
8  };
9  var addAmericano = function (name) {
10     coffeeList += ', ' + name;
11     console.log(coffeeList);
12     setTimeout(addMocha, 500, '카페모카');
13 };
14 var addMocha = function (name) {
15     coffeeList += ', ' + name;
16     console.log(coffeeList);
17     setTimeout(addLatte, 500, '카페라떼');
18 };
19 var addLatte = function (name) {
20     coffeeList += ', ' + name;
21     console.log(coffeeList);
22 };
23
24 setTimeout(addEspresso, 500, '에스프레소');
25
```

콜백 지옥과 비동기 제어

Promise

- ES6

```
1
2 new Promise(function (resolve) {
3     setTimeout(function () {
4         var name = '에스프레소';
5         console.log(name);
6         resolve(name);
7     }, 500)
8 }).then(function (prevName) {
9     return new Promise(function (resolve) {
10         setTimeout(function () {
11             var name = prevName + ', 아메리카노';
12             console.log(name);
13             resolve(name);
14         }, 500);
15     });
16 }).then(function (prevName) {
17     return new Promise(function (resolve) {
18         setTimeout(function () {
19             var name = prevName + ', 카페모카';
20             console.log(name);
21             resolve(name);
22         }, 500);
23     });
24 }).then(function (prevName) {
25     return new Promise(function (resolve) {
26         setTimeout(function () {
27             var name = prevName + ', 카페라떼';
28             console.log(name);
29             resolve(name);
30         }, 500);
31     });
32 });
33
```

콜백 지옥과 비동기 제어



● 반복적인 내용을 함수화

```
1
2  var addCoffee = function (name) {
3      return function (prevName) {
4          return new Promise(function (resolve) {
5              setTimeout(function () {
6                  var newName = prevName ? prevName + ', ' + name : name;
7                  console.log(newName);
8                  resolve(newName);
9              }, 500);
10         });
11     };
12 };
13
14 addCoffee('에스프레소')()
15     .then(addCoffee('아메리카노'))
16     .then(addCoffee('카페모카'))
17     .then(addCoffee('카페라떼'));
18
```


콜백 지옥과 비동기 제어



● Generator

- ES6

```
1
2  var addCoffee = function (prevName, name) {
3      |   setTimeout(function () {
4      |       |   coffeeMaker.next(prevName ? prevName + ', ' + name : name);
5      |   }, 500);
6  };
7
8  var coffeeGenerator = function* () {
9      |   var espresso = yield addCoffee('', '에스프레소');
10     |   console.log(espresso);
11     |   var americano = yield addCoffee(espresso, '아메리카노');
12     |   console.log(americano);
13     |   var mocha = yield addCoffee(americano, '카페모카');
14     |   console.log(mocha);
15     |   var latte = yield addCoffee(mocha, '카페라떼');
16     |   console.log(latte);
17  };
18
19  var coffeeMaker = coffeeGenerator();
20  coffeeMaker.next();
21
```


콜백 지옥과 비동기 제어

● async/await

- ES2017

비동기 작업을 수행하고자 하는 함수 앞에 async를 표기

함수 내부에서 비동기 작업이 필요한 위치마다 await를 표기

Promise로 자동 전환하고 해당 내용이 resolve 된 이후에야 다음으로 진행

```
1
2  var addCoffee = function (name) {
3      return new Promise(function (resolve) {
4          setTimeout(function () {
5              resolve(name);
6          }, 500);
7      });
8  };
9
10 var coffeeMaker = async function () {
11     var coffeeList = '';
12     var _addCoffee = async function (name) {
13         coffeeList += (coffeeList ? ',' : '')
14             + (await addCoffee(name));
15     };
16
17     await _addCoffee('에스프레소');
18     console.log(coffeeList);
19     await _addCoffee('아메리카노');
20     console.log(coffeeList);
21     await _addCoffee('카페모카');
22     console.log(coffeeList);
23     await _addCoffee('카페라떼');
24     console.log(coffeeList);
25 };
26
27 coffeeMaker();
28
```

정리



● 콜백 함수는 다른 코드에 인자로 넘겨줌으로써 그 제어권도 함께 위임한 함수

- 콜백 함수를 호출하는 시점을 결정
- 콜백 함수를 호출할 때 인자로 넘겨줄 값들 및 그 순서를 결정
- 콜백 함수의 `this`가 무엇을 바라보도록 할지가 정해져 있는 경우도 있음
 - `bind` 함수로 변경 가능
- 어떤 함수에 인자로 메소드를 전달하더라도 결국 함수로 실행
- 비동기 제어를 위해 콜백 함수를 사용하다보면 콜백 지옥에 빠지기 쉬움
 - `Promise`, `Generator`, `async/await`와 같은 방법을 사용하면 콜백 지옥에서 벗어날 수 있음